

Промежуточный отчет по программному проекту

1. Основные планы и этапы проекта

1.1 Краткое описание проекта:

«База геномных вариантов DBG» – это веб-приложение для хранения, анализа и визуализации геномных вариантов. Приложение обеспечивает удобный доступ к данным о пациентах и их геномных вариантах, загрузку файлов VCF, CSV и FASTQ, запуск биоинформатического пайплайна и обогащение данных из публичных источников.

Название проекта:

«База геномных вариантов DBG»

Цель проекта:

Разработать функциональное веб-приложение для хранения, анализа и визуализации геномных вариантов с поддержкой биоинформатического пайплайна и автоматического обогащения данных из публичных источников (MyVariant.info, ClinVar). Это станет решением для врачей-генетиков и исследователей, нуждающихся в удобном инструменте для управления геномными данными пациентов.

Краткое описание задач:

1. Разработка технического задания, определение требований, определение архитектуры проекта.
2. Разработка серверной части (backend) на языке Go с REST API.
3. Разработка клиентской части (frontend) на React с использованием TanStack Table и TanStack Virtual.
4. Настройка базы данных PostgreSQL и Docker-контейнеризации.
5. Интеграция биоинформатического пайплайна (bwa, samtools, bcftools).
6. Разработка программной документации: написание технического задания, пояснительной записки, текста программы, руководства оператора.
7. Испытания программы.
8. Защита проекта.

1.2 Планы и этапы выполнения проекта

Этап проекта	Описание работ	Ожидаемые результаты	Сроки выполнения
Обоснование необходимости разработки	Постановка задачи и сбор исходных теоретических материалов	Поставлены задачи и собрана литература и источники для теоретического решения задач	13.11.25
Научно-исследовательский этап разработки	Определение структуры входных и выходных данных; Предварительный выбор методов решения задач;	Определение требований для решения задачи	15.11.25 – 03.12.25

Этап проекта	Описание работ	Ожидаемые результаты	Сроки выполнения
	Определение требований к техническим и программным средствам; Обоснование возможности решения поставленной задачи		
Разработка и утверждение технического задания	Определение требований к программному продукту; Выбор языков программирования; Разработка и согласование технического задания с научным руководителем; Загрузка согласованного технического задания в SmartLMS	Готовое техническое задание для начала разработки приложения	15.11.25 – 04.12.25
Разработка программы	Разработка серверной части на Go; Разработка клиентской части на React; Настройка PostgreSQL и Docker; Интеграция биоинформатического пайплайна	Готовое приложение, реализующее все функции, определённые в техническом задании	05.12.25 – 31.01.26
Разработка программной документации	Разработка документов в соответствии с требованиями ГОСТ 19 ЕСПД (Единой системы программной документации)	Готовая программная документация	05.12.25 – 31.01.26
Испытания программы	Разработка, согласование и утверждение порядка и методики испытаний; Проведение испытаний программы в соответствии с утверждённым порядком и методикой; Корректировка программы и программной документации по результатам испытаний;	Тестирование программы и исправление недочётов	01.02.26 – 28.02.26
Подготовка и передача программы	Подготовка программы и программной документации для презентации и защиты; Представление разработанного программного продукта научному руководителю и получение отзыва; Загрузка Пояснительной записки в систему Антиплагиат через ЛМС НИУ ВШЭ; Загрузка материалов курсового про-	Получение готового продукта, реализующего все функции, и защита работы	01.03.26 – 01.05.26

Этап проекта	Описание работ	Ожидаемые результаты	Сроки выполнения
	екта в ЛМС, дисциплина «Курсовой проект, 2 курс, ПИ»; Защита программного продукта комиссии		

2. Используемый технологический стек и его обоснование

2.1 Перечень используемых технологий

Технология/Инструмент	Описание	Причины выбора
Go (Golang)	Язык программирования для разработки серверной части (backend)	Высокая производительность, встроенная конкурентность, простота развёртывания
React 19	JavaScript-библиотека для создания пользовательского интерфейса (frontend)	Компонентный подход, обширная экосистема, поддержка виртуального скроллинга
PostgreSQL 14+	Реляционная база данных	Поддержка JSONB, enum-типов, представлений и триггеров
Docker	Платформа контейнеризации	Изоляция среды, воспроизводимость, установка биоинформатических инструментов
Vite 6	Сборщик frontend-приложения	Высокая скорость сборки и горячей перезагрузки (HMR)
bwa / samtools / bcftools	Биоинформатические инструменты	Стандартные инструменты для выравнивания ридов и вызова вариантов
Git	Управление версиями	Удобство разработки, сохранение версий

2.2 Обоснование выбранного технологического стека

Использование языка программирования **Go** для серверной части имеет следующие преимущества:

1. Высокая производительность: Go компилируется в нативный код, что обеспечивает быструю обработку запросов и эффективное использование ресурсов.
2. Конкурентность: Встроенная поддержка горутин позволяет эффективно обрабатывать множество одновременных запросов и задач пайплайна.

3. Простота развёртывания: Go компилируется в единый статический бинарный файл, что упрощает контейнеризацию.
4. Стандартная библиотека: net/http предоставляет всё необходимое для создания HTTP-сервера без внешних зависимостей.

Использование **React 19** для клиентской части обусловлено:

1. Компонентный подход: Позволяет создавать переиспользуемые элементы интерфейса.
2. Экосистема: Интеграция с TanStack Table 8 и TanStack Virtual 3 для эффективной работы с большими таблицами.
3. Context API: Удобное управление состоянием аутентификации и данных приложения.
4. React Router DOM 7: Клиентская маршрутизация для SPA.

PostgreSQL выбран, потому что:

1. Поддержка сложных типов данных (JSONB, enum), необходимых для хранения геномных аннотаций.
2. Высокая производительность при работе с большими объёмами данных.
3. Поддержка представлений (views) и триггеров для автоматизации.

Docker используется, так как:

1. Обеспечивает изоляцию среды и воспроизводимость.
2. Упрощает установку биоинформатических инструментов (bwa, samtools, bcftools).
3. Docker Compose позволяет запустить всю систему одной командой.

3. Критерии оценивания проекта

Критерий	Описание
Использование адаптивного дизайна	Будет использовано/Не будет использовано
Функциональность – Процент выполнения функциональных требований	Выполненные требования в процентах от общего количества
Функциональность – Количество реализованных функций	Абсолютное количество функций, которые работают правильно
Документация и оформление – Полнота документации (%)	Процент от требуемого объёма документации
Документация и оформление – Процент закомментированных строк (%)	Процент закомментированных строк относительно общего количества строк кода
Соблюдение сроков и плана – Процент выполнения работы в срок (%)	Процент задач, выполненных в срок
Соблюдение сроков и плана – Количество дней отклонения от плана	Общее число дней отклонения от плана
Использование технологического стека - Процент использования функциональности стека (%)	Процент использования функциональности выбранного стека технологий

Критерий	Описание
Оценка командной работы – Среднее время коммуникации (в часах)	Среднее время, потраченное на обсуждение задач и решение вопросов
Оценка командной работы – Количество завершённых задач на каждого участника	Общее число задач, выполненных каждым членом команды